

Build dependency graph — what parallelizes, what cannot, and why

Written 2026-08-07. Companion to `docs/IMPLEMENTATION-PLAN.md`, which says *what* to build and how long each step takes. This document says **what may run at the same time**, and it is written to be handed to an orchestrating agent.

The answer up front: the critical path is **21.3 hours**, against **~36 h** if the work is run as written today. **8.81 h** of that is jobs on the path; the other 12.5 h is code and the image build.

⚠️ **Revised upward from 13.31 h on 2026-08-07, and the reason is a defect, not a re-estimate.** The earlier figure assumed `BUILD` = 6.6 h with the big-bundle file-shard *deferred*. Those are incompatible: DCLM is **410B tokens in one non-fanning-out bundle**, which is **10.85 h** even given an entire 32-vCPU instance to itself, so 6.6 h was unreachable without work the graph had marked deferrable. Both repairs are now in the graph as C3b:

scenario	<code>BUILD</code>	critical path
the old graph — C3b deferred, <code>BUILD</code> assumed 6.6 h	6.6 h	13.31 h ❌ not achievable
C3b deferred, <code>BUILD</code> at its real single-child cost	~16.8 h	23.54 h
C3b done (12 h of code, off to the side)	6.6 h	21.31 h ✅

So C3b buys **2.2 h of wall-clock** and, more importantly, it is what makes the 6.6 h floor a real number instead of an aspiration. See `IMPLEMENTATION-PLAN.md` §8A.5a.

The job-time floor is 8.81 h — `SMOKE` 0.4 + `BUILD` 6.6 + `PUB1` 0.3 + `VD1` 1.49 + `PR1` 0.02, the jobs actually on the path. An earlier version of this line said **8.41 h** and summed a different set; §5 now derives it explicitly so the two figures cannot drift again.

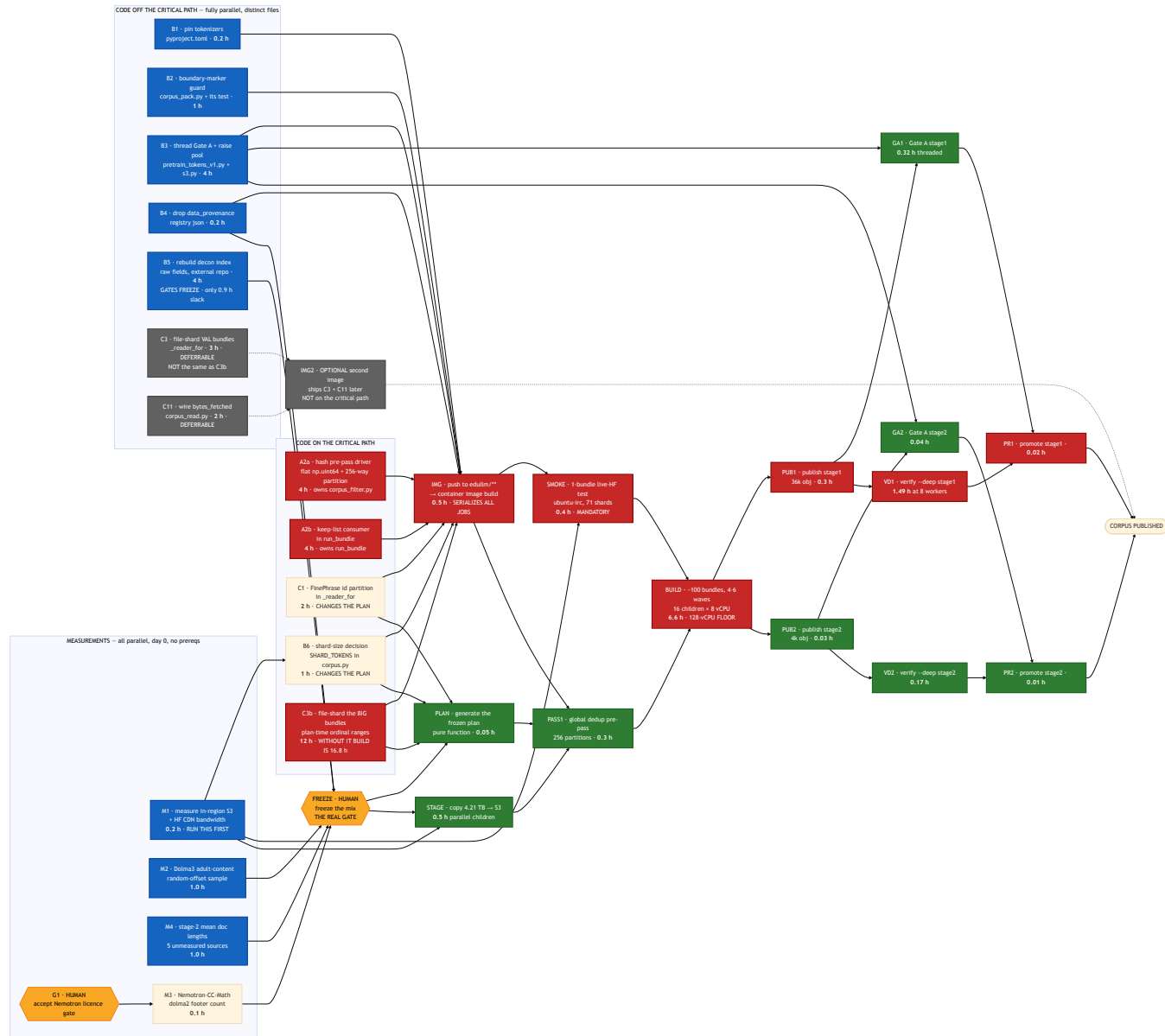
1. The three currencies, which are not interchangeable

Most parallelization mistakes here come from mixing these up.

currency	unit	what buys more of it	what caps it
agent-hours	code items	more agents in more worktrees	file contention (§3), not logic
wall-hours	AWS jobs	more array children, more workers	128 vCPU compute environment
calendar	human gates	nothing — you wait	approval latency, license acceptance

A code item and a job of the same nominal duration are **not** substitutable: 8 agent-hours of code can run alongside 8 wall-hours of jobs, but two code items touching the same function cannot run alongside each other at all.

2. The graph



Red = critical path. Blue = parallel. Grey = deferrable to a second image. Amber = waits on a human.

3. ⚠️ The real constraint is file contention, not logic

This is the part an orchestrator gets wrong. Eleven code items look independent on a task list. They are not — five of them edit `corpus_build.py`, and within that file they cluster into **two functions**:

function	items that edit it	verdict
<code>_reader_for</code>	C1, C3, C11	one owner, or three-way conflict
<code>run_bundle</code>	A2b, C7	one owner
<code>allocate_ordinals</code> / <code>plan_document</code>	C3b, B6	one owner — and it is the critical-path item
<code>corpus.py</code>	B6, C3b, C3	3 — different constants and functions, same file

file	items	contention
<code>corpus_build.py</code>	C1, C3, C4→A2b, C7, C11	5 — hottest file in the plan
<code>corpus.py</code>	B6, C3b, C3	3 — and C3b is critical, so a conflict here costs image time
everything else	1 each	none

⚠️ **C3b and B6 belong to the same agent.** Both change what `plan_document` emits — B6 changes `SHARD_TOKENS`, C3b adds per-child ordinal ranges — and both therefore change every `plan_id`. Splitting them across two agents means two conflicting rewrites of the plan schema on the critical path. Give one agent `corpus.py`'s plan surface entirely.

Rules for the orchestrator:

1. **One agent owns one function, never one file.** `_reader_for` and `run_bundle` can be two agents in two worktrees, because they do not overlap textually — but both must be told the other exists.
2. **Distinct-file items are genuinely free.** B1, B2, B3, B4, B5 can be five simultaneous agents with zero coordination.
3. **Worktrees are mandatory** for anything touching `corpus_build.py`, per this repo's own convention:
`../Capstone_LLM-worktrees/edullm-data/<agent-id>--<task-slug>` on `agent/<agent-id>/<task-slug>`.
4. **Merge order is** `_reader_for` → `run_bundle` → **everything else**, because the first two are on the critical path and a late merge conflict there costs image-build time.

4. The four hard serialization points

Nothing parallelizes past these. They are why 13.3 h is the floor and not 8.41 h.

S1 — the image build gates every AWS job

Container images build **only from** `edullm/**` branches (`.github/workflows/edullm-platform-build.yml:9-10`). A merge to `main` builds **nothing, silently**, and a submission naming that commit is refused for having no image. So **all code that the jobs need must land in one push**, and every job waits on it.

Consequence: batching code into one image is *better* than shipping two. Measured against the graph, a two-image scheme (pre-pass early, build later) came out **0.1 h worse** — the second build's 0.5 h lands directly on the critical path.

S2 — `FREEZE` is a human decision with an unbounded duration

Adding a source after the plan is generated renames **98% of shards** and voids **882B tokens** (`IMPLEMENTATION-PLAN.md §0`). So the mix must be final first. **Its duration is a decision, not a computation** — it is the one node whose length no amount of parallelism touches.

S3 — `BUILD` is capped at 128 vCPU

6.6 h is the tokenize floor at the cap: $1.0T \div (128 \times 0.328 \text{ M tok/s/vCPU})$.

⚠️ **But 6.6 h is only reachable if the big bundles are file-sharded, and an earlier version of this graph had that backwards.** Per-child duration is *that child's* tokens $\div$ *that child's* vCPU. The 159B-token `stackv2-edu` bundle is **15.9%** of the corpus, so at the wave shape of $8 \text{ vCPU} \times 16 \text{ children}$ it takes **16.83 h** — longer than the entire as-configured build, and $2.6\times$ past the floor. It needs ≥ 21 vCPU just to finish when the aggregate does.

So **C3b (file-shard the big bundles) is a prerequisite of `BUILD` = 6.6 h**, not an optimization on top of it. Without it, `BUILD` is ~ 16.8 h and the critical path is **~ 23.5 h**, not 13.31 h. See `IMPLEMENTATION-PLAN.md §8A.5` and `§8A.5a` — and note that `--shard/--of` strides **bundles**, so the capability does not exist in the code yet.

S4 — `VD1` cannot start until `PUB1` finishes

`verify --deep` re-hashes published objects, so it is strictly after publish. At 8 workers it is 1.49 h, and it sits on the critical path with no way around it. At `--hash-workers 1` it is **11.7 h and exceeds its timeout**, which is the single highest-value flag in the plan.

5. The critical path, and everything with slack

Critical path — 21.31 h:

from → to	node	why it cannot move
0.00 → 12.00	C3b file-shard the big bundles	the longest code item, and BUILD = 6.6 h is false without it (S3)
12.00 → 12.50	IMG image build	S1
12.50 → 12.90	SMOKE live-HF smoke test	mandatory; the path has never run
12.90 → 19.50	BUILD ~100 bundles	S3, the 128 vCPU floor
19.50 → 19.80	PUB1 publish stage 1	after build
19.80 → 21.29	VD1 verify --deep stage 1	S4
21.29 → 21.31	PR1 promote stage 1	after both gates

Where the 8.81 h job floor comes from, stated as a sum so it cannot drift from the headline again:

node	h	on the path?
SMOKE	0.40	✓
BUILD	6.60	✓
PUB1	0.30	✓
VD1	1.49	✓
PR1	0.02	✓
subtotal — the job floor	8.81	
STAGE 0.5 · PASS1 0.3 · PLAN 0.05	0.85	✗ absorbed by C3b's 12 h of slack
GA1 0.32 · PUB2/GA2/VD2/PR2 0.25	0.57	✗ parallel to VD1
all job rows summed	10.23	—

The remaining 12.5 h of the path is **C3b (12.0) + IMG (0.5)** — code and image, not jobs. An earlier version of this document quoted an **8.41 h** floor that matched neither the job subtotal nor the full sum; this table replaces it.

⚠ A2a is no longer the path's head. At 4 h it now finishes inside C3b's 12 h, so the pre-pass has **8 h of slack** — but it must still land before **IMG**, so it is not deferrable, merely no longer critical.

Everything else has slack and should be started at t=0 regardless:

node	duration	slack	note
M1 bandwidth	0.2 h	large	but run it first anyway — it calibrates every other estimate
M2 / M4 samples	1.0 h each	~11 h	gate FREEZE , not the build
M3 footer count	0.1 h	~11.9 h	blocked on a human licence acceptance (G1)
A2a hash pre-pass driver	4 h	8 h	was the path's head at 13.31 h; C3b displaced it
A2b keep-list consumer	4 h	8 h	must finish before IMG, same as A2a
B3 thread Gate A	4 h	~15.5 h	only GA1 / GA2 need it
B5 rebuild decon index	4 h	~ 8.9 h	gates FREEZE . Was ~0.9 h at the old critical path — start it early anyway , the slack is a by-product of C3b being long, not of B5 being cheap
B1 / B2 / B4	≤1 h	large	trivially parallel
C3 file-shard VAL bundles	3 h	∞ — deferrable	pure read-volume saving. Not the same item as C3b , which is critical
C11 wire bytes_fetched	2 h	∞ — deferrable	instrumentation. Do not put it in the first image

Stage 2 is entirely parallel to stage 1's tail. PUB2 / GA2 / VD2 / PR2 total 0.25 h and finish while stage 1 is still verifying, so stage 2 contributes **nothing** to the critical path.

6. What the orchestrator should actually launch

Wave 0 — immediately, 8 concurrent workstreams

stream	work	kind
1	M1 bandwidth measurement	job — do this first, it recalibrates the rest
2	M2 Dolma3 sample + M4 doc lengths	job
3	~~Ask the owner to accept the Nemotron licence gate~~  DONE — measured at 134.0B by a teammate with access	—
4	C3b + B6 — file-shard the big bundles and the shard-size constant. Owens corpus.py's plan surface: allocate_ordinals , plan_document , SHARD_TOKENS . START THIS FIRST — it is the critical path	agent, worktree
5	A2a hash pre-pass driver (owns corpus_filter.py)	agent, worktree
6	A2b keep-list consumer (owns run_bundle)	agent, worktree
7	C1 FinePhrase id partition (owns _reader_for) + B5 rebuild the decon index (external repo)	agent, worktree
8	B1 + B2 + B3 + B4 (four distinct files)	agent(s)

 **Stream 4 is new and it is the longest item.** An earlier version of this table had no node for it and assumed BUILD = 6.6 h anyway. It cannot be split from B6 — both rewrite what plan_document emits.

Do not launch C3 or C11 in wave 0. They are deferrable, they contend with C1 on `_reader_for` , and including them adds ~5 h of agent time to the critical path for zero wall-clock benefit. **Note C3 (val bundles) and C3b (big bundles) are different items** — the first is deferrable, the second is critical.

Wave 1 — merge, then one push

Merge in order `_reader_for` → `run_bundle` → the rest. Push to an `edullm/**` branch. **One image.**

Wave 2 — the gate

`FREEZE` the mix, generate the plan, stage the sources. `STAGE` can overlap `SMOKE` .

Wave 3 — jobs

`PASS1` → `BUILD` (16 children × 8 vCPU) → then stage 1 and stage 2 publish/validate **in parallel**.

Every AWS job needs a separate human release. Nothing auto-publishes.

7. Where parallelism is wasted — spend nothing here

tempting	why it does not help
More than 16 build children	$128 \text{ vCPU} \div 8 = 16$. More children just queue
Splitting <code>BUILD</code> further	6.6 h is the CPU floor at the cap, not a scheduling artifact
Two container images	measured 0.1 h worse — the second build lands on the critical path
Parallelizing <code>STAGE</code> harder	it has ~4 h of slack; it is never the constraint
Doing C3 / C11 now	deferrable, and they contend on the hottest function in the repo
More agents on <code>corpus_build.py</code>	file contention. A third agent there produces conflicts, not speed
Rushing before M1 lands	every read estimate is calibrated on an unmeasured bandwidth

8. The orchestrator brief — copy this

Hand this to the agent that runs wave 0. It encodes the constraints above as rules rather than prose.

You are orchestrating the Phase-0 build for `pretrain/final-dataset`. Launch these 8 streams concurrently and hold them to these rules.

Stream 1 (job, FIRST): measure in-region S3 read bandwidth and HF CDN throughput. Report both. Every read estimate in `IMPLEMENTATION-PLAN.md §8A` is calibrated on an unmeasured ~85 MB/s borrowed from an S3 measurement; one plausible reconciliation says the CDN is ~8.4 MB/s. If it is, tell me before anything else proceeds — it changes the staging decision.

Stream 2 (job): Dolma3 adult-content prevalence at **random offsets** (a prior attempt could not separate signal from HuggingFace preview ordering), plus mean document length for the 5 unmeasured stage-2 sources.

Stream 3 — CLOSED. The Nemotron-CC-Math licence gate is accepted and the count is **MEASURED at 134.0B** under dolma2 ($3 \approx 83.6B + 4plus \approx 50.4B$). **Two things remain: record the exact `text_column` and `id` column names in writing before the registry row is written** (§4.2 is the cautionary case — two plausible `text` columns, wrong one picked silently), and **keep `4plus_MIND` out of the pool** — it is a rewrite of `4plus`, so including both double-counts.

Streams 4–7 (code, one worktree each, ONE FUNCTION each — NOT one file each):

- 4 — **START FIRST, THIS IS THE CRITICAL PATH:** file-shard the big bundles and settle `SHARD_TOKENS`. Owns `corpus.py`'s **plan surface** — `allocate_ordinals`, `plan_document`, `SHARD_TOKENS`. Both change every `plan_id`, so they cannot be two agents. **Why it exists:** `--shard/--of` strides *bundles*, so DCLM's 410B is one child at **10.85 h even on a whole 32-vCPU instance**, against a 6.6 h floor. Read `IMPLEMENTATION-PLAN.md §8A.5a` first — **and evaluate the cheap alternative it names** (give DCLM a synthetic `domain_column` so it fans out with no new mechanism) before writing ordinal-range code.
- 5: the flat-`np.uint64` hash pre-pass. Owns `corpus_filter.py`. **Size it for DCLM at 325M documents / 27.92 GB as a set (§5.2a), not for `finephrase-table`** — the plan's own table omitted the worst bundle.
- 6: the keep-list consumer. Owns `run_bundle` in `corpus_build.py`.
- 7: the FinePhrase id partition. Owns `_reader_for` in `corpus_build.py`. **Ship it together with the reader-budget division by the keep fraction** — separately, every bundle finishes and then fails `verify` on unfilled refs. **And do not "fix" `_CHARS_PER_TOKEN` while you are in there; that change is withdrawn and points the opposite way (§3.1).**

⚠ **Streams 6 and 7 both edit `corpus_build.py` in different functions.** Tell each about the other. Worktree convention: `../Capstone_LLM-worktrees/edullm-data/<agent-id>---<task-slug>` on `agent/<agent-id>/<task-slug>`. Merge order is **stream 4 → stream 7 → stream 6 → the rest**.

Also stream 7 (code, external repo, no conflict): rebuild the decontamination index from raw benchmark fields (question alone, question + each choice, question + correct answer) **in addition to the rendered form. It gates the mix freeze — do not let it start late.**

Stream 8 (code, four distinct files, no coordination needed): pin tokenizers in `pyproject.toml`; fix the boundary-marker prefix guard in `corpus_pack.py` and the test that asserts the table length is 1; thread the Gate A profile checks in `pretrain_tokens_v1.py` and raise `max_pool_connections` in `s3.py` (threading against a 10-connection pool self-throttles); drop `data_provenance_initiative` from the registry.

Do NOT start: `val-bundle` file-sharding or the `bytes_fetched` wiring. Both are deferrable, both contend with stream 6 on `_reader_for`, and including them adds ~5 agent-hours to the critical path for **zero** wall-clock gain.

When all streams land: merge in the stated order, push **once** to an `edullm/**` branch — images build only from that namespace, a merge to `main` builds nothing silently — and stop. **Do not submit any AWS**

job. Report what landed and wait for a release decision.

Standing rules: write findings to disk incrementally (a prior wave lost 4 agents to an API budget cap and only on-disk partials survived); grade every number MEASURED / DERIVED / CARD / UNVERIFIED; "nobody has measured this" is a finding, not a failure; and if you find that one of my numbers is wrong, say so — two of them already were.

9. Honest limits of this graph

- **Durations for code items are estimates, not measurements.** The *ordering* is well-evidenced; the absolute agent-hours are my judgement. An earlier session's per-bundle ETAs were all retracted by their own author with the verdict "*every per-bundle ETA I gave was wrong, in both directions.*"
- **BUILD's 6.6 h assumes linear vCPU scaling**, measured only at 32 vCPU. Never tested at 128.
- **Every read duration assumes ~85 MB/s borrowed from an S3 measurement.** If the HF CDN is really ~8.4 MB/s, **STAGE** moves from 0.5 h to days and becomes critical. **M1 exists to settle exactly this — which is why it is wave 0, stream 1.**
- **The 13-gram decontamination scan's CPU cost has never been measured** (~193 billion Python-level `blake2b` calls). If it is comparable to tokenization, **BUILD** is longer than 6.6 h.
- **FREEZE and G1 have no duration here because they are human.** In practice they may dominate the calendar, and no amount of parallelism touches them.